

Containerized Application Deployment using Kubernetes, Amazon ECR, Amazon EKS and AWS Fargate

1. Introduction

This project demonstrates the containerization and deployment of the FluxOps web application using Docker, Kubernetes, Amazon Elastic Container Registry (ECR), Amazon Elastic Kubernetes Service (EKS), and AWS Fargate.

FluxOps is a web-based DevOps command-learning application built using React, Vite, TypeScript, and Tailwind CSS. The application was containerized using Docker and deployed first on a local Kubernetes environment using Minikube and then on Amazon EKS using AWS Fargate.

2. Objectives

The main objectives of this project were:

- To containerize the FluxOps application using Docker.
 - To deploy the application on a local Kubernetes cluster.
 - To verify the application locally using Kubernetes.
 - To store the Docker image in Amazon ECR.
 - To create and configure an Amazon EKS cluster.
 - To configure AWS Fargate for serverless Kubernetes workloads.
 - To deploy the application on EKS using the ECR image.
 - To expose the application using an AWS Network Load Balancer.
 - To verify external accessibility of the application.
 - To troubleshoot deployment and AWS permission issues.
 - To implement Git and GitHub version control.
-

3. Application Selection and Preparation

The application selected for this project was FluxOps.

FluxOps is a DevOps command-learning web application that provides useful commands and information related to DevOps technologies.

Technologies Used

- React
- Vite
- TypeScript
- Tailwind CSS
- Docker
- Nginx
- Kubernetes
- Minikube
- Amazon ECR
- Amazon EKS
- AWS Fargate
- Git
- GitHub

The application source code was obtained from the GitHub repository:

<https://github.com/saad7804/fluxops>

The project dependencies were installed using:

```
npm install
```

The production application was built using:

```
npm run build
```

The build completed successfully and generated the production files in the dist directory.

4. Docker Containerization

A multi-stage Dockerfile was used to containerize the FluxOps application.

The first stage used Node.js to install dependencies and build the React application.

The second stage used Nginx to serve the production build.

The Docker image was created with the following name:

```
fluxops:v1
```

The image was successfully built and verified using Docker commands.

The production container uses Nginx to serve the application on port 80.

5. Local Kubernetes Environment Setup

Minikube was selected as the local Kubernetes environment.

The Minikube cluster was started using the Docker driver:

```
minikube start --driver=docker
```

The Kubernetes node was verified using:

```
kubectl get nodes
```

The node successfully reached the Ready state.

The Docker image was loaded into Minikube using:

```
minikube image load fluxops:v1
```

6. Application Deployment on Local Kubernetes

The FluxOps application was deployed to Minikube using Kubernetes Deployment and Service configuration files.

The Deployment used the local Docker image:

```
fluxops:v1
```

The image pull policy was configured so that Kubernetes could use the locally loaded image.

The application was exposed using a Kubernetes NodePort Service.

The application was accessed using the Minikube service URL.

The application returned the FluxOps web page successfully.

This confirmed that the containerized application was working correctly on local Kubernetes.

7. Kubernetes Verification and Troubleshooting

The local Kubernetes deployment was verified using:

kubectl get nodes

kubectl get pods

kubectl get deployment

kubectl get service

kubectl logs deployment/fluxops

The application Pods were running successfully.

The Deployment showed the expected number of replicas.

The Service was available and the application could be accessed through the Minikube service URL.

Basic troubleshooting was performed by checking:

- Pod status
- Deployment status
- Service status
- Application logs
- Application accessibility

The application responded successfully, confirming that the local Kubernetes deployment was working correctly.

8. Amazon ECR Implementation

An Amazon Elastic Container Registry repository named fluxops was created in the AWS ap-south-1 region.

The ECR repository was:

```
888869353635.dkr.ecr.ap-south-1.amazonaws.com/fluxops
```

The local Docker image was tagged for Amazon ECR:

```
888869353635.dkr.ecr.ap-south-1.amazonaws.com/fluxops:v1
```

The image was then pushed to Amazon ECR.

The image was verified in the Amazon ECR repository with the tag:

```
v1
```

This confirmed that the application container image was successfully stored in Amazon ECR.

9. Amazon EKS Cluster Setup

An Amazon EKS cluster named:

```
fluxops-cluster
```

was created in the AWS ap-south-1 region.

The cluster was configured with AWS Fargate support.

The EKS cluster was verified using:

```
kubectl cluster-info
```

and:

```
kubectl get nodes
```

The cluster became active successfully and Kubernetes communication with the cluster was established.

10. AWS Fargate Configuration

A Kubernetes namespace named fluxops was created:

```
kubectl create namespace fluxops
```

A dedicated Fargate profile named:

```
fluxops-fargate
```

was created for the fluxops namespace.

The Fargate profile was verified using:

```
eksctl get fargateprofile --cluster fluxops-cluster --region ap-south-1
```

The FluxOps application Pods were successfully scheduled on AWS Fargate.

This allowed the application to run without managing traditional EC2 worker nodes for the application workload.

11. Application Deployment on Amazon EKS

The FluxOps application was deployed to Amazon EKS using the Docker image stored in Amazon ECR.

The deployment configuration included:

- Application name: FluxOps
- Namespace: fluxops
- Replicas: 2
- Container port: 80
- Image source: Amazon ECR
- Runtime: AWS Fargate

The ECR image used by the Deployment was:

```
888869353635.dkr.ecr.ap-south-1.amazonaws.com/fluxops:v1
```

Two application replicas were successfully created.

The Pods showed:

READY STATUS

1/1 Running

1/1 Running

The Pods were running on Fargate nodes.

12. Application Exposure on Amazon EKS

A Kubernetes LoadBalancer Service was configured to expose the FluxOps application externally.

The AWS Load Balancer Controller was configured for the EKS cluster.

Because the application was running on Fargate, the Load Balancer was configured to use IP targets.

The Service used:

- Service type: LoadBalancer
- Port: 80
- Target port: 80
- Load Balancer: AWS Network Load Balancer
- Scheme: Internet-facing
- Target type: IP

The Service successfully received an AWS Load Balancer DNS name.

13. EKS Application Verification

The EKS application was verified using multiple Kubernetes commands.

Cluster verification

```
kubectl cluster-info
```

This confirmed that the Kubernetes control plane was accessible.

Fargate verification

```
eksctl get fargateprofile --cluster fluxops-cluster --region ap-south-1
```

This confirmed that the required Fargate profile was configured.

Pod verification

```
kubectl get pods -n fluxops -o wide
```

The result showed two FluxOps Pods in the Running state.

Both Pods had:

READY 1/1

RESTARTS 0

Deployment verification

```
kubectl get deployment fluxops -n fluxops
```

This confirmed that the Deployment was available with the required replicas.

Service verification

```
kubectl get service fluxops-service -n fluxops
```

This confirmed that the application Service was exposed using an AWS Load Balancer.

Application logs

Application logs were checked using:

```
kubectl logs deployment/fluxops -n fluxops
```

The logs were accessible and no application-level failure prevented the Nginx web server from running.

Complete resource verification

The Kubernetes resources were also checked using:

```
kubectl get all -n fluxops
```

This displayed the Deployment, ReplicaSet, Pods, and Service.

14. External Accessibility Verification

The AWS Network Load Balancer provided the following DNS name:

```
k8s-fluxops-fluxopss-4cce754727-5f30becbba14bd2.elb.ap-south-1.amazonaws.com
```

External accessibility was verified using:

```
curl -I http://k8s-fluxops-fluxopss-4cce754727-5f30becbba14bd2.elb.ap-south-1.amazonaws.com
```

The application successfully returned:

```
HTTP/1.1 200 OK
```

```
Server: nginx/1.31.6
```

```
Content-Type: text/html
```

This confirmed that the FluxOps application was successfully accessible through the external AWS Load Balancer.

15. Troubleshooting and Resolution

During the EKS deployment, the LoadBalancer Service initially remained in the Pending state.

Problem

The AWS Load Balancer Controller reported an IAM authorization error related to:

`ec2:CreateSecurityGroup`

The IAM role used by the AWS Load Balancer Controller did not have the required EC2 permissions.

Investigation

The Service status and events were checked using:

```
kubectl describe service fluxops-service -n fluxops
```

The events showed that the controller was unable to create the required security group because of insufficient IAM permissions.

Resolution

The required EC2 permissions were attached to the IAM role used by the AWS Load Balancer Controller.

The Service was then reconciled using:

```
kubectl annotate service fluxops-service -n fluxops force-reconcile="$(date +%s)" --overwrite
```

After reconciliation, the AWS Network Load Balancer was successfully provisioned.

The Service received an external DNS name and the application became externally accessible.

Final Result

The application returned:

HTTP/1.1 200 OK

Therefore, the IAM issue was successfully resolved and the EKS application became accessible externally.

16. Git and GitHub Implementation

Git was used for version control throughout the project.

The project repository was:

<https://github.com/saad7804/fluxops>

The Kubernetes and EKS configuration files were added to the repository.

Important configuration files include:

deployment.yml

service.yml

eks-deployment.yml

eks-service.yml

Dockerfile

README.md

The changes were committed using:

Add EKS and Fargate deployment configuration

The changes were pushed to the main branch on GitHub.

The Git working tree was verified after pushing the changes.

17. Project Structure

The important project structure is:

fluxops/

- |— Dockerfile
- |— README.md
- |— deployment.yml
- |— service.yml
- |— eks-deployment.yml
- |— eks-service.yml
- |— nginx.conf
- |— package.json
- |— package-lock.json
- |— src/
- |— dist/
- └— documentation/
 - └— Project_Report.md

18. Final Outcome

The FluxOps application was successfully:

1. Prepared and built.
2. Containerized using Docker.
3. Deployed on local Kubernetes using Minikube.
4. Verified on local Kubernetes.
5. Stored in Amazon ECR.
6. Deployed on Amazon EKS.
7. Configured to run on AWS Fargate.
8. Exposed through an AWS Network Load Balancer.
9. Verified externally using HTTP.
10. Version controlled using Git and GitHub.

The final external accessibility test returned:

HTTP/1.1 200 OK

This confirms that the containerized FluxOps application was successfully deployed and made accessible through Amazon EKS and AWS Fargate.

19. Conclusion

This project provided practical experience with containerization, Kubernetes, Amazon ECR, Amazon EKS, AWS Fargate, AWS Load Balancing, IAM permissions, troubleshooting, and GitHub.

The project demonstrated an end-to-end DevOps deployment workflow starting from application preparation and Docker containerization and continuing through local Kubernetes testing, container image storage in Amazon ECR, EKS deployment, Fargate execution, external application exposure, troubleshooting, and GitHub version control.

The successful HTTP 200 OK response from the AWS Load Balancer confirmed that the application was deployed and accessible successfully.